

Brief – API Gestion Pédagogique

Contexte

Une école souhaite développer une **API de gestion pédagogique** permettant d'organiser les cours, gérer les inscriptions des étudiants et faciliter la participation aux activités académiques.

En tant que développeur, vous devez concevoir une **API Laravel** permettant :

- la gestion des cours
- les inscriptions des étudiants
- la recommandation de cours
- l'organisation des groupes de travail
- la gestion des paiements

L'architecture devra respecter :

- **Repository Pattern**
 - **Service Layer**
 - **Authentification JWT**
 - **Tests unitaires**
 - **Documentation API**
-

Jour 1 — Initialisation du projet et architecture

Objectif

Mettre en place le projet Laravel et préparer l'architecture.

Étapes techniques

1. Créer le projet Laravel

Créer un nouveau projet Laravel.

Configurer :

- base de données
- environnement `.env`

Lancer les migrations par défaut.

2. Préparer l'architecture

Créer les dossiers nécessaires :

- `app/`
- `Repositories/`
- `Services/`
- `Interfaces/`

Objectif :

- Repository → accès base de données
 - Service → logique métier
 - Controller → gestion des requêtes API
-

3. Créer les modèles principaux

Identifier les entités nécessaires :

- User
- Course
- Enrollment
- Group
- Interest
- Wishlist

Créer les migrations correspondantes.

Guide :

- réfléchir aux **relations entre les tables**
 - définir les **clés étrangères**
-

Jour 2 — Authentification JWT

Objectif

Mettre en place une authentification sécurisée.

Étapes techniques

Installer une librairie JWT pour Laravel.

Configurer :

- génération de token
- protection des routes

Créer un **AuthController**.

Endpoints attendus :

- register
- login
- logout
- reset password

Guide d'implémentation :

- lors de l'inscription → créer l'utilisateur
 - générer un token lors du login
 - protéger les routes avec middleware JWT
-

Rôles

Lors de l'inscription :

- role = student
- role = teacher

Ce rôle ne doit **jamais être modifiable** après création.

Guide :

- ajouter une colonne **role** dans **users**
- utiliser un middleware pour vérifier les permissions

Jour 3 — Gestion des cours

Objectif

Permettre aux enseignants de gérer leurs cours.

Étapes techniques

Créer la table `courses`.

Colonnes possibles :

- title
- description
- price
- teacher_id

Relation :

- Course belongsTo User (teacher)
-

Fonctionnalités API

Teacher peut :

- créer un cours
- modifier un cours
- supprimer un cours

Student peut :

- consulter les cours
- voir les détails

Guide :

- vérifier le rôle dans les controllers
- utiliser un service pour la logique métier

Jour 4 — Repository Pattern

Objectif

Séparer l'accès aux données de la logique métier.

Créer :

- CourseRepositoryInterface
- CourseRepository

Le repository doit contenir :

- create
- update
- delete
- findById
- getAll

Guide :

- le controller ne doit **jamais accéder directement au modèle**
 - utiliser l'interface dans le service
-

Jour 5 — Recommandation de cours

Objectif

Proposer des cours selon les intérêts des étudiants.

Créer la table :

- interests

Créer une relation :

- users ↔ interests
- courses ↔ interests

Guide :

- un étudiant choisit ses intérêts lors de l'inscription
- récupérer les cours liés à ces intérêts

Exemple logique :

- `courses where interest_id IN user_interests`

La logique doit être dans un **Service**.

Jour 6 — Wishlist

Objectif

Permettre aux étudiants de sauvegarder des cours.

Créer la table :

- wishlists

Colonnes :

- user_id
- course_id

Fonctionnalités :

- ajouter un cours en favoris
- supprimer un favori

- afficher la wishlist

Guide :

- vérifier que l'utilisateur est un étudiant
 - éviter les doublons dans la wishlist
-

Jour 7 — Inscription aux cours

Objectif

Permettre aux étudiants de participer aux cours.

Créer la table :

- enrollments

Colonnes possibles :

- user_id
- course_id
- payment_status

Fonctionnalités :

- s'inscrire à un cours
- se retirer d'un cours

Guide :

- vérifier que l'utilisateur est étudiant
 - empêcher l'inscription multiple au même cours
-

Jour 8 — Paiement Stripe

Objectif

Gérer le paiement pour l'inscription aux cours.

Étapes :

- créer un service Stripe
- générer une session de paiement
- confirmer le paiement

Guide :

1. l'étudiant clique sur **payer**
 2. Stripe crée une session
 3. après paiement → créer l'inscription
-

Jour 9 — Organisation des groupes

Objectif

Répartir automatiquement les étudiants.

Créer la table :

- groups

Colonnes :

- course_id
- group_number

Règle :

Un groupe = **25 étudiants maximum**

Guide de logique :

- chercher le dernier groupe du cours
- si groupe plein
- créer un nouveau groupe
- sinon

- ajouter l'étudiant au groupe

Cette logique doit être dans un **GroupService**.

Jour 10 — Tests et documentation

Objectif

Finaliser l'API.

Tests unitaires

Créer des tests pour :

- inscription
- création cours
- inscription étudiant
- logique groupes

Utiliser :

- PHPUnit
ou
 - Pest
-

Tests Postman

Tester :

- authentification
 - CRUD cours
 - wishlist
 - inscription
 - paiement
-

Documentation API

Créer une documentation avec :

- Postman
- Swagger

La documentation doit inclure :

- endpoint
 - méthode HTTP
 - paramètres
 - exemple de réponse
-

Résultat attendu

L'API finale doit permettre :

- authentification JWT
- gestion des cours
- recommandation basée sur les intérêts
- wishlist
- inscription aux cours
- paiement Stripe
- gestion automatique des groupes
- tests unitaires
- documentation API complète